

# Guía completa: Hola Mundo en Node.js con TypeScript

Este documento describe **todos los pasos**, desde cero, para crear un proyecto **Node.js + TypeScript**, ejecutar un *Hola Mundo*, configurar correctamente TypeScript y evitar errores comunes, incluyendo las **correcciones realizadas** durante el proceso. También incluye la configuración recomendada de **Visual Studio Code**.

---

## 1. Requisitos previos

Antes de empezar, es necesario tener instalado:

- **Node.js (versión LTS)**

Descargar desde: <https://nodejs.org>

Comprobar la instalación:

```
node -v  
npm -v
```

---

## 2. Creación del proyecto

### 2.1 Crear carpeta del proyecto

```
mkdir nodejs-prueba  
cd nodejs-prueba
```

### 2.2 Inicializar proyecto Node.js

```
npm init -y
```

Esto crea el archivo `package.json`.

---

## 3. Instalación de TypeScript y dependencias

Instalamos las dependencias necesarias **solo para desarrollo**:

```
npm install -D typescript ts-node @types/node
```

**Función de cada uno:** - `typescript`: compilador TypeScript - `ts-node`: permite ejecutar `.ts` directamente - `@types/node`: tipos de Node.js para TypeScript

---

## 4. Configuración de TypeScript

### 4.1 Crear archivo `tsconfig.json`

```
npx tsc --init
```

### 4.2 Configuración correcta de `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

#### Corrección importante aplicada

Se añadió explícitamente: - `include: ["src"]`

Esto evita errores como:

File is not under 'rootDir'

que ocurre cuando TypeScript intenta compilar archivos de configuración como `eslint.config.mts`.

---

## 5. Estructura del proyecto

Crear la carpeta del código fuente:

```
mkdir src
```

Crear el archivo principal:

**src/index.ts**

```
console.log("Hola mundo desde TypeScript y Node.js 🙌");
```

## 6. Ejecución del proyecto

### 6.1 Modo desarrollo (sin compilar)

```
npx ts-node src/index.ts
```

Salida esperada:

```
Hola mundo desde TypeScript y Node.js 🙌
```

### 6.2 Compilación (build)

```
npx tsc
```

Esto genera el código JavaScript en la carpeta `dist/`.

### 6.3 Ejecución en producción

```
node dist/index.js
```

## 7. Scripts NPM recomendados

Editar `package.json` y añadir:

```
"scripts": {  
  "dev": "ts-node src/index.ts",  
  "build": "tsc",  
  "start": "node dist/index.js"  
}
```

### Uso de cada script

- `npm run dev` → desarrollo (TypeScript directo)
- `npm run build` → compilar TypeScript a JavaScript
- `npm start` → ejecutar el código compilado

---

## 8. Visual Studio Code (VS Code)

### 8.1 Editor recomendado

Se utiliza **Visual Studio Code** como editor principal.

---

### 8.2 Extensiones imprescindibles

Instalar las siguientes extensiones:

1. **TypeScript and JavaScript Language Features** (incluida por defecto)
  2. **ESLint** (Microsoft)
  3. **Prettier - Code formatter**
- 

### 8.3 Extensiones muy recomendadas

1. **Node.js Modules Intellisense**
  2. **Path Intellisense**
  3. **Error Lens**
- 

### 8.4 Extensiones opcionales

1. **GitLens** (integración avanzada con Git)
- 

## 9. Añadir Nodemon (recarga automática en desarrollo)

### 9.1 ¿Qué es Nodemon?

**Nodemon** es una herramienta que reinicia automáticamente la aplicación Node.js cada vez que detecta cambios en los archivos. Es ideal para desarrollo.

---

### 9.2 Instalación de Nodemon

Instalar como dependencia de desarrollo:

```
npm install -D nodemon
```

---

### 9.3 Configuración con TypeScript

Modificar el script `dev` en `package.json`:

```
"scripts": {  
  "dev": "nodemon --exec ts-node src/index.ts",  
  "build": "tsc",  
  "start": "node dist/index.js"  
}
```

Ahora, al ejecutar:

```
npm run dev
```

la aplicación se reiniciará automáticamente al guardar cambios.

---

## 9.4 (Opcional) Archivo `nodemon.json`

Para mayor control:

```
{  
  "watch": ["src"],  
  "ext": "ts",  
  "ignore": ["dist"],  
  "exec": "ts-node src/index.ts"  
}
```

---

## 10. Integrar ESLint + Prettier (paso a paso)

### 10.1 ¿Para qué sirven?

- **ESLint**: detecta errores y malas prácticas
  - **Prettier**: formatea el código automáticamente
- 

### 10.2 Instalación de dependencias

```
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin prettier eslint-config-prettier eslint-plugin-prettier
```

---

### 10.3 Inicializar ESLint

```
npx eslint --init
```

Seleccionar: - TypeScript - Node.js - Estilo de módulos CommonJS - Archivo de configuración: JavaScript o ESM

---

## 10.4 Configuración básica de ESLint

Ejemplo de `eslint.config.mts`:

```
import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';

export default tseslint.config(
  eslint.configs.recommended,
  ...tseslint.configs.recommended,
  {
    ignores: ['dist']
  }
);
```

---

## 10.5 Configurar Prettier

Crear archivo `.prettierrc`:

```
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "es5"
}
```

---

## 10.6 Integración ESLint + Prettier

Prettier se integra mediante: - `eslint-config-prettier` - `eslint-plugin-prettier`

Esto evita conflictos entre reglas.

---

## 10.7 Scripts útiles

Añadir a `package.json`:

```
"scripts": {
  "lint": "eslint src",
```

```
"lint:fix": "eslint src --fix"
}
```

## 10.8 Integración con VS Code

Instalar extensiones: - ESLint - Prettier

Configurar `settings.json`:

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "eslint.validate": ["typescript"],
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  }
}
```

## 11. Estructura final del proyecto

```
nodejs-prueba/
├── src/
│   └── index.ts
├── dist/
│   └── index.js
├── eslint.config.mts    (si existe, no se compila)
├── package.json
├── tsconfig.json
└── node_modules/
```

## 10. Flujo de trabajo recomendado

1. Desarrollo:

```
npm run dev
```

2. Compilación:

```
npm run build
```

### 3. Producción:

```
npm start
```

---

## 11. Estructura final del proyecto

```
nodejs-prueba/  
├─ src/  
│   └─ index.ts  
├─ dist/  
│   └─ index.js  
├─ eslint.config.mts  
├─ nodemon.json (opcional)  
├─ .prettierrc  
├─ package.json  
├─ tsconfig.json  
└─ node_modules/
```

---

## 12. Conclusión

Con esta configuración: - El proyecto está correctamente estructurado - TypeScript solo compila el código fuente real - Se evitan errores comunes de `rootDir` - El flujo desarrollo → build → producción está bien definido

Este proyecto sirve como **base profesional** para cualquier aplicación Node.js con TypeScript.